Save Point

The Convex backend

QuillFin · Prepared for Logan Young, Save Point Limited · 6 November 2026

`backend/` is the Convex project both QuillFin apps talk to: the Flutter app (over Convex's HTTP API) and the Next.js web app (Phase 4, over Convex's React client). It holds the synced copy of each user's data, their plan and usage, the AI generation action on QuillFin's own key, and Stripe billing. The design is in `quillfin_plan.md` §2-3; this document is the reference for what was built.

Layout

```
backend/
  convex/
    schema.ts          tables and indexes
    validators.ts      the synced tables' row shapes (mirror lib/storage/db.dart)
    plans.ts           every plan and sync limit, in one place
    users.ts           identity helpers; users:ensure, users:me
    sync.ts            sync:push, sync:pull
    rows.ts            RowWriter: every write to a synced table (seq, row counts, tombstones,
cascades)
    folders.ts, notes.ts, questions.ts, problems.ts, today.ts
                       the web app's data functions (see "The web app's functions")
    scheduling.ts      FSRS, a direct port of the Dart scheduler the app uses
    grading.ts         short-answer grading, ported from lib/core/grading.dart
    billing.ts         billing:me; usage reservation; subscription bookkeeping
    ai.ts              ai:generateQuestions
    aiPrompts.ts       prompt and parser, ported from lib/ai/
    stripeActions.ts   Checkout, Billing Portal, webhook handling (Node runtime)
    stripeSync.ts      Stripe objects -> stored subscription (pure, tested)
    http.ts            POST /stripe/webhook
    auth.config.ts     Clerk issuer and the two token audiences
    _generated/        written by `convex dev`; committed
  tests/               vitest + convex-test, one file per module
```

Running it

```
npm install                 # from the repo root (npm workspaces)
npm run backend:test        # 131 tests
npm run coverage --workspace backend
npm run backend:typecheck
```

With no Convex account, `npx convex dev` in `backend/` can run a local anonymous deployment (`CONVEX_AGENT_MODE=anonymous npx convex dev`). Set the two auth variables to placeholders first or the push is refused:

```
cd backend
npx convex env set CLERK_JWT_ISSUER_DOMAIN https://placeholder.clerk.accounts.dev
npx convex env set CLERK_OAUTH_CLIENT_ID placeholder
```

The real deployment is set up in `setup_guide.md`.

Environment variables

All set on the Convex deployment (`npx convex env set NAME value`), never in the repo or an app.

Variable	Needed for	Value
`CLERK_JWT_ISSUER_DOMAIN`	everything (required to deploy)	Clerk Frontend API URL, e.g. `https://xyz.clerk.accounts.dev`
`CLERK_OAUTH_CLIENT_ID`	Flutter sign-in (required to deploy)	client id of the Clerk OAuth application
`ANTHROPIC_API_KEY`	AI generation	QuillFin's Anthropic key
`QUILLFIN_AI_MODEL`	optional	model id; defaults to `claude-sonnet-5-5`
`STRIPE_SECRET_KEY`	billing	`sk_test_…` / `sk_live_…`
`STRIPE_WEBHOOK_SECRET`	billing	`whsec_…` of the `/stripe/webhook` endpoint
`STRIPE_PRICE_PRO_MONTHLY`	billing	monthly Pro price id
`STRIPE_PRICE_PRO_YEARLY`	billing	yearly Pro price id
`SITE_URL`	billing	web app origin, e.g. `https://quillfin.app`

Anything unset fails with a clear error code (`AI_NOT_CONFIGURED`, `BILLING_NOT_CONFIGURED`) instead of crashing, and `billing:me` reports `upgradeAvailable: false` until a Pro price is set.

Identity

Every public function reads the caller from `ctx.auth.getUserIdentity()` and keys the user by its `subject`, the Clerk user id. That is the same whether the token came from the web app (Clerk session JWT, template `convex`) or the Flutter app (Clerk OAuth ID token), so one person is one `users` row. Users are created lazily by their first mutation; apps call `users:ensure` after sign-in to also store the email and name.

Sync

Row shape

Each synced table stores an envelope around `data`:

Field	Meaning
`uid`	client-generated UUID, unique per user per table
`updatedAt`	last-writer-wins clock, ms since epoch
`deletedAt`	set on a tombstone; `data` is then dropped
`seq`	the user's sync counter when this row last changed
`data`	the row itself

`data` mirrors one row of the local SQLite table in camelCase. Parent foreign keys become the parent's `uid` (`folderUid`, `noteUid`, `questionUid`, `problemUid`, `parentFolderUid`), 0/1 flags become booleans, and domain timestamps stay ISO-8601 strings exactly as stored locally. Nullable columns accept either `null` or a missing field. The tables are `folders`, `notes`, `questions`, `questionChoices`, `questionHistory`, `problems` and `problemAttempts`.

`sync:push({ changes })`

`changes` maps table name to a list of `{ uid, updatedAt, deletedAt?, data? }`.

- At most 200 rows per call, across all tables.
- Each row is applied only if its `updatedAt` is **newer** than the stored one. Equal or older is skipped, so retrying a push is harmless.
- `updatedAt` (and `deletedAt`) are clamped to the server's clock, so a device whose clock runs ahead can't win every conflict.
- `questionHistory` and `problemAttempts` are append-only: once a row exists, only a delete applies to it.
- A tombstone drops `data`. A tombstone for a row the server never saw is still stored, so other devices learn about the delete.
- Returns `{ applied, skipped }`.

The whole batch is rejected (nothing written) on the first bad row:

code	When
`BATCH_TOO_LARGE`	more than 200 rows
`INVALID_ROW`	empty or >64-char `uid`, or no `data` and not deleted; has `table`, `uid`
`NOTE_TOO_LARGE`	a note body over 200 KB (UTF-8); has `table`, `uid`
`ROW_LIMIT`	a new live row past 50,000 in one table; has `table`

`sync:pull({ since, limit? })`

Returns `{ changes, cursor, hasMore }`: rows with `seq > since`, oldest first, including tombstones (`data: null`), at most `limit` rows (default 200, max 500). The client applies them, stores `cursor`, and calls again while `hasMore`. `seq` is unique per user, so paging never skips a row. A user who has never pushed gets `{ changes: {}, cursor: since, hasMore: false }`.

What the client must do (Phase 3)

- Tombstone children itself when it deletes a parent. SQLite's cascades don't reach the server.
- When applying a pulled row, skip it if the local copy is still dirty and newer. Otherwise the pulled row wins.
- Treat a pulled row whose parent `uid` isn't known yet as pending. A page can contain a child before its parent arrives.

The web app's functions

The web app (`webapp.md`) reads and writes Convex directly. Its functions all act on the signed-in user's rows only, and all writes go through `RowWriter` (`rows.ts`), the same path `sync:push` uses. So every web edit gets the next `seq` and reaches the apps on their next pull, row limits and the note size limit apply, and deletes become tombstones.

Module	Functions
`folders`	`list`, `create`, `rename`, `remove` (with subfolders, notes, their questions)
`notes`	`listByFolder`, `get`, `create`, `update`, `remove` (with its questions)
`questions`	`listByNote`, `due`, `saveGenerated`, `update`, `setActive`, `remove`, `record`
`problems`	`list`, `get`, `create`, `update`, `remove`, `logAttempt`
`today`	`summary` (due counts, today's reviews and attempts, problems due, 7-day forecast)

- Rows the web creates get the same 32-hex `uid` form as the app's, and the same field values. For example, a new question is a fresh FSRS card due now, and tags are trimmed, lower-cased and de-duplicated.
- **Removing on the web cascades on the server**: the app deletes children by `ON DELETE CASCADE` and pushes their tombstones, but the web has to tombstone them itself.

- `questions:record` **and** `problems:logAttempt` **schedule on the server** with `scheduling.ts`, at the server's time, and add the history row. The browser grades answers itself (`grading.ts`) so feedback is instant.
- Queries that depend on the time (`questions:due`, `today:summary`) take `now` from the client, so their results stay cached between ticks; the client also sends the start of its local day.
- `schema.ts` adds indexes for these: rows by parent uid, questions and problems by `nextDueAt`, history by `askedAt`, attempts by `attemptedAt`.

FSRS parity

`scheduling.ts` ports the Dart `fsrs` 2.0.1 package's `reviewCard` as the app configures it: the package's default weights, learning steps of 1 and 10 minutes, a relearning step of 10 minutes, and no fuzzing. It's a port rather than `ts-fsrs` because that library's defaults and step handling differ, and a review must come out the same on any device. `tests/fixtures/fsrs_parity.json` holds 36 rating sequences at three target recalls, all run through the app's own scheduler by `tool/fsrs_parity_fixture.dart`. The tests replay them and require the same state, step, stability, difficulty (to 1e-9), and due time to the millisecond. Regenerate the fixture after any change to the app's scheduling:

```
dart run tool/fsrs_parity_fixture.dart > backend/tests/fixtures/fsrs_parity.json
```

Plans and usage

`plans.ts` holds the limits: Free 20 and Pro 500 AI generations a month on QuillFin's key. The plan is **Pro** while the user has a subscription in `active`, `trialing` or `past_due` status. That status only ever comes from Stripe, through the webhook.

`billing:me` returns everything Settings > Plan needs:

```
{
  plan: "free" | "pro",
  planName: string,
  aiNotes: { period: "2026-10", used: number, limit: number },
  subscription: { status, interval, currentPeriodEnd, cancelAtPeriodEnd } | null,
  upgradeAvailable: boolean,
}
```

Usage resets each UTC calendar month.

AI generation

`ai:generateQuestions({ noteText, numQuestions, questionFormat, linkedNotes? })` returns `{ questions, usage }`, where `questions` has the same shape as the Flutter app's `GeneratedQuestion`. The prompt and parser are ports of `lib/ai/prompts.dart` and `parseGeneratedQuestions`, plus tolerance for a Markdown code fence around the JSON.

It first reserves one generation from the allowance (in a mutation, so two parallel requests can't both take the last one), then calls Anthropic. If the call fails, the generation is refunded.

code	When
`INVALID_REQUEST`	empty note, over 60,000 characters, or not 1-20 questions
`AI_NOT_CONFIGURED`	no `ANTHROPIC_API_KEY`
`AI_LIMIT_REACHED`	allowance used up; has `plan`, `limit`, `period`
`AI_UNAVAILABLE`	network failure or an HTTP error from Anthropic
`AI_BAD_RESPONSE`	the reply wasn't the expected JSON

Users who use their own key keep calling their provider directly from the app. That path never touches the backend or the allowance.

Billing

- `stripeActions:createCheckoutSession({ interval: "month" | "year" })` returns `{ url }` for Stripe Checkout. It creates the Stripe customer on first use (with `userId` and `clerkId` metadata) and tags the subscription with `userId`. Throws `ALREADY_PRO` if the user is already Pro.
- `stripeActions:createPortalSession()` returns `{ url }` for the Billing Portal. Throws `NO_BILLING_ACCOUNT` before the first checkout.
- Both return the browser to `SITE_URL/settings/plan`. The return URL is fixed server-side so Checkout can't be used as an open redirect.
- `POST /stripe/webhook` verifies the `Stripe-Signature` header, then for checkout-completed and every `customer.subscription.*` event re-reads the subscription from Stripe and stores that copy. Events arriving out of order therefore can't leave a stale status, and replays are harmless. A bad signature or a failure answers 400, and Stripe retries.

The webhook URL is the deployment's **site** URL: `https://<deployment>.convex.site/stripe/webhook`.

Not built yet

- **Account deletion** (Phase 5): removing a user's rows needs batched, scheduled deletes (up to 350,000 rows), plus cancelling Stripe and deleting the Clerk user.
- **Rebuilding FSRS cards from history** after two offline devices review the same card. Last-writer-wins keeps the later review, and both history rows survive (`quillfin_plan.md` §3).